

第2章. 资料进入检索系统：Loader、Embedding 与 Milvus

资料怎样变成可检索的数据？

学习目标

1. 读取与校验 CSV，解释一行一条 Document 的原因。
2. 区分检索文本、元数据、Embedding 与索引。
3. 理解 Dense 检索、HNSW 与 COSINE 的协作。
4. 按源码阅读顺序完成离线实验，再准备索引服务。

阅读方法：先看业务问题和图，再读名词与源码，最后复制完整实验验证。代码节选用于定位实现；完整实验会明确运行条件。

代码基准：2026-10-03 本地项目当前源码。默认配置与环境覆盖值可能不同；本文不以旧实施计划代替真实实现。

运行约定：命令从项目根目录执行。学习实验代码直接收录在 PDF 内；自行新建练习文件后复制运行，不另附脚本。服务实验的结果应由你实际运行记录。

学习产出：每章至少留下一份自己的输入、输出和解释。涉及检索质量、耗时或简历中的改善幅度，只采用有证据的数据。

本章目录

2.1 需求分析：为什么先研究资料，再研究模型 3

2.2 名词解析：从原始记录到可检索数据 3

2.3 认识源数据：ID、层级路径与标题 4

2.4 Loader 实现：先校验，再转换，再汇总 5

2.5 Document 构造：检索文字与显示信息各自承担职责 6

2.6 为什么本项目没有先使用 Text Splitter 7

2.7 Embedding：模型怎样把文字变成可比较的向量 7

2.8 Milvus：Collection、主键与索引参数 9

2.9 Dense 与 Hybrid：同一份资料，两种检索线索 11

2.10 写入策略：新增、更新与重建的差别 11

2.11 离线实验：只读取 CSV，观察转换结果 12

2.12 离线实验：用替身观察 Dense Retriever 的接口 13

2.13 源码阅读与本章检查清单 14

2.14 排查练习：先确定问题发生在哪一层 14

本章从项目真实的 CSV 出发，学习资料怎样变成可检索的 Document，再理解 Embedding 和 Milvus 的职责。本章沿着真实资料的转换过程讲解，并通过源码定位与小实验检查理解。所有命令均在项目根目录执行。

2.1 需求分析：为什么先研究资料，再研究模型

用户输入“仪表针脚图”，系统需要返回真实存在的资料名称和分类路径。假如直接让对话模型生成文件名，它可能给出一个听起来合理、资料库里却不存在的名称。因此，项目首先需要建立可靠的资料入口：读出原始记录，保留稳定 ID，形成统一的检索文本。

这里的“资料”有明确边界。当前 data/circuit-data.csv 保存资料 ID、层级路径和关联文件名称；它没有保存电路图中的连线、针脚表内容或故障诊断结论。本章建立的是“找到资料条目”的能力。后续若需要回答某个针脚连接哪里，需要增加实际文件内容的读取与证据抽取，不能只靠标题检索补出这些细节。

先提出三个问题：一条数据是否能稳定对应一个资料条目？检索文字是否包含用户可能描述的品牌、设备和资料类型？结果能否追溯回原始资料？这三个问题决定 Loader 的设计，之后才轮到使用哪个向量模型。

用户需求	项目现在提供的基础	还需要的数据
查找三一仪表相关资料	标题与分类路径的检索	当前 CSV 已提供入口
显示资料 ID 和所在分类	metadata 中的稳定字段	保持原始 ID 与路径一致
解释某个针脚的电压或用途	仅凭标题无法确认	原始图纸、表格或正文证据
自动诊断设备故障	当前未提供诊断依据	电路知识、检测结果及安全边界

本节小结：先定义资料单位和输出边界，可以避免把“找到一份图纸”误写成“理解了这份图纸”。这个区分也决定学习笔记和简历里的描述是否准确。

2.2 名词解析：从原始记录到可检索数据

Loader（加载器） 负责读文件、检查字段并转换对象。它解决输入格式问题，不负责判断用户的问题应该命中哪一份资料。

Document（文档对象） 是 LangChain 常用的数据容器。page_content 保存用于检索的文本，metadata 保存 ID、标题、分类等辅助信息。Document 可以代表整篇文章、一个切块或一个业务记录，具体由项目决定。

Embedding（向量表示） 把文字变成一组数值，让程序能够计算语义上的接近程度。**Vector Store（向量存储）** 保存文字、向量与元信息，并提供检索接口。项目用 Milvus 承担这个存储职责。

Ingestion（数据入库） 指从原始数据到可查询索引的准备过程。它与在线检索是两条流程：准备阶段可以批量处理全量资料；用户查询阶段只处理一个问题并返回有限候选。

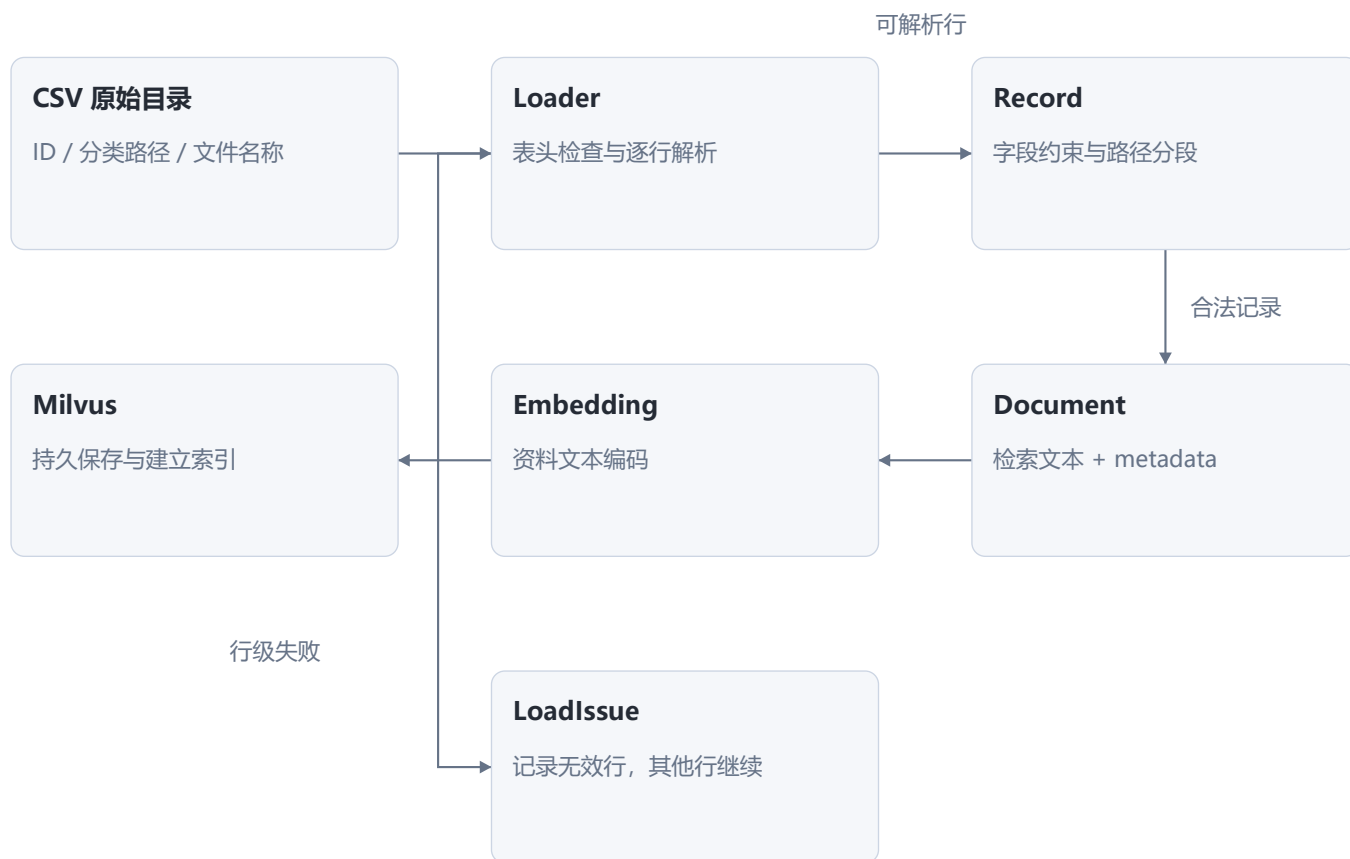


图 2-1. CSV 先转换为统一 Document，模型编码与数据库写入属于资料准备阶段；问题查询使用已准备好的索引。

不要把这张图理解为所有节点每次查询都会执行。读取整份 CSV 和批量写入发生在准备阶段；用户查询通常连接已经存在的 Collection。源数据发生变化后，是否更新索引，需要独立的维护策略。

读源码时可以沿着上述职责逐个定位：Loader 在 `app/rag/loader.py`；Embedding 在 `app/rag/embeddings.py`；Dense 的集合与索引定义在 `app/rag/vectorstore.py`；Retriever 在 `app/rag/retriever.py`。先找到每层负责的输入输出，再进入实现细节。

易错点：Document 是程序对象，Collection 是存储集合，Index 是查找结构。它们不是三种同义的“知识库”，分别回答数据表达、持久保存和检索加速三个问题。

2.3 认识源数据：ID、层级路径与标题

需求是让一条资料在转换和检索后仍然能被明确识别。项目用三个必需 CSV 表头：ID、层级路径、关联文件名称。表头拼错时，Loader 会直接报错；某一行的值不合法时，会收集为问题记录并继续处理其他行。

`app/schemas/document.py` 中的 `CircuitDocumentRecord` 是规范化后的记录结构。doc_id 要求大于 0，标题与层级路径不能是空字符串。hierarchy_segments 按 -> 切分路径，对每一级去掉两侧空格，再去掉空的片段。

本次在 2026-10-03 只读检查当前 CSV：共 4233 条非空记录，转换成功 4233 条，无效记录 0 条；4233 个 ID 全部唯一。这是对本地文件的检查结果，不代表 Milvus 里当前也有同样数量的记录。数据库是否同步，需要另行查询验证。

字段	第一条真实记录	学习时关注什么
ID	1	原始身份，不能用检索排名替代
层级路径	电路图→ECU电路图→工程机械→三一→德国 仪表	同时带有业务类别与品牌线索
关联文件名称	三一挖掘机_德国仪表显示器针脚定义	给用户展示，也参与检索

这条记录的最后一级是“德国仪表”。它不是一个由统一业务 Schema 保证的“车型”字段。路径深度、每一级的含义可能因资料而变化，所以学习时应该称为层级分类，而不是随意固定为“品牌、车型、年份”。

源码节选：以下保留当前字段校验的核心内容，用于阅读，不是独立启动命令。

代码块 python

```
1 class CircuitDocumentRecord(BaseModel):
2     doc_id: int = Field(gt=0)
3     hierarchy_path: str = Field(min_length=1)
4     title: str = Field(min_length=1)
5
6     @property
7     def hierarchy_segments(self) -> list[str]:
8         return [
9             segment.strip()
10            for segment in self.hierarchy_path.split("->")
11            if segment.strip()
12        ]
```

易错点：当前 Loader 检查字段合法性，没有显式拒绝重复 ID。如果未来源数据出现重复 ID，本地 doc_id -> Document 字典可能覆盖前面的记录，向量写入也会共享主键。应把唯一性作为独立数据质量检查，而不是认为 Pydantic 自动做了全表检查。

2.4 Loader 实现：先校验，再转换，再汇总

加载器采用一个可读的顺序：检查文件是否存在，读取 CSV 表头，逐行解析，构造 Document，最后返回有效数据和问题列表。文件层面的错误终止加载；行层面的错误只影响对应记录。这样的设计适合批量资料准备，也方便定位具体失败行。

CSV 使用 utf-8-sig 编码读取，可以兼容带 BOM 的 UTF-8 文件。csv.DictReader 将每行映射成“表头→值”的字典。记录原始行号从 2 开始，因为第 1 行是表头；报错中的行号可以直接用于回到文件检查。

完全空白的行会跳过，不计入 total_rows。对于进入统计的记录，理想关系是 total_rows = valid_rows + invalid_rows。valid_rows 来自 documents 长度，invalid_rows 来自 issues 长度，所以不要把总行数理解成文本编辑器显示的物理行数。

实现思路：校验不是为了给数据贴标签，而是阻止错误对象继续流入模型和数据库。标题为空时，把它变成向量没有业务意义；ID 不合法时，即使检索命中也无法稳定回查资料。

源码节选：这是解析失败的处理方式，完整实现见 app/rag/loader.py。

代码块 python

```
1 raw_id = (row.get(ID_COLUMN) or "").strip()
2 hierarchy_path = (row.get(HIERARCHY_COLUMN) or "").strip()
3 title = (row.get(TITLE_COLUMN) or "").strip()
4
5 if not raw_id or not hierarchy_path or not title:
6     return None, LoadIssue(
7         row_number=row_number,
8         message="ID、层级路径、关联文件名称均不能为空",
9         raw=dict(row),
10    )
```

输入问题	当前处理	为什么这样处理
CSV 不存在	FileNotFoundError	没有数据源，无法继续
必需表头缺失	ValueError	全文件结构不符合约定
某行缺少必需值	收集 LoadIssue	保留问题位置，其他行继续
ID 不是整数或不大于 0	收集 LoadIssue	防止无效身份进入结果
空白行	跳过	避免将排版空行当成资料

应用场景：以后学习商品、法律条文或企业 FAQ 项目时，可以复用这种“文件错误终止、行错误收集”的判断方式。但字段规则应该来自业务数据，不能机械套用电路图的三个表头。

2.5 Document 构造：检索文字与显示信息各自承担职责

本项目的 build_search_text() 把路径片段用空格连接，再拼上资料名称。这样同一条 Document 同时包含“工程机械”“三一”“德国仪表”“针脚定义”等线索。用户没有完整说出标题时，检索仍然有机会找到相关条目。

record_to_document() 还保存五类 metadata: doc_id、title、hierarchy_path、hierarchy_segments、hierarchy_depth。其中路径字符串保留原始表达，片段列表方便分类计算，深度反映分类层级数量。

源码节选：这一函数给 Dense 与 Sparse 检索提供统一的文本入口。

代码块 python

```
1 def build_search_text(record: CircuitDocumentRecord) -> str:
2     normalized_path = " ".join(record.hierarchy_segments)
3     return (
4         f"层级路径: {normalized_path}\n"
5         f"关联文件名称: {record.title}"
6     )
```

预期对象解释：第一条记录的 page_content 是两行文字，第一行列出规范化路径，第二行列出文件名称。metadata['doc_id'] 是整数 1，hierarchy_depth 是 5。向量模型接收的是 page_content，不是把所有

metadata 自动拼进去。

Dense 与 Hybrid 的 `_prepare_document()` 在入库前会再整理 metadata，仅保留 ID、标题、路径和深度。`hierarchy_segments` 没有按当前整理函数写入向量库。因而不能仅凭 Loader 中有这个字段，就断言每个 Milvus 结果也会携带片段列表。

这里有一个值得迁移的阅读习惯：追踪字段经过每层转换后的形态。输入结构、内存对象、数据库字段和 API 响应不一定完全一致。只看定义而不看转换函数，往往会误以为某个字段始终存在。

本节小结：检索文本帮助系统找到记录；metadata 帮助系统解释、过滤、展示和追溯记录。改变其中一类信息后，要检查另一类是否仍然匹配，尤其是标题和 ID。

2.6 为什么本项目没有先使用 Text Splitter

飞书 RAG 课程会讲 Text Splitter（文本切分器）：把长文拆成小块，便于检索相关段落并控制上下文大小。这个思路很有用，但当前项目的原始数据已经是一条条短资料记录，所以采用“一行 CSV 对应一个 Document”。

这不是遗漏了 RAG 的固定步骤。切分的目标是形成合理检索单位，而不是让所有项目都出现一个 splitter 类。若把一个很短的资料标题拆成几个小片段，可能反而丢失品牌、设备类型与标题之间的联系，还会增加结果去重和原记录回查的工作。

数据形态	合理检索单位	需要重点保留的关联
当前 CSV 资料目录	一条资料记录	稳定 ID、标题与分类路径
长篇维修手册	段落或小节	文档 ID、页码、章节、邻接块
电路图上的表格	一张表或合理行组	图号、页码、表头、设备型号
扫描图纸或图片	OCR / 图像理解后再设计	原图位置、识别置信度及来源

需求变化时怎么办：如果以后加入维修手册 PDF，先判断文本提取是否可靠，再按章节或语义结构切块，并为每块增加来源元信息。资料目录仍可以保留为入口，与正文块建立父子关联，不必立即用正文切块替代全部目录记录。

易错点：当前 `Document.page_content` 中出现“针脚定义”这几个字，只能证明文件名称包含这个主题，不能证明其中有某个具体针脚的定义。理解这个限制，才不会在学习示例里把目录当成全文证据。

检查题：如果向资料库增加一份 100 页的维修手册，能否仍然只保存文件名称？可以用于查找手册，但无法可靠回答其中的细节问题。要回答细节，应把正文中可引用的内容作为检索对象，并保留页码等定位信息。

2.7 Embedding：模型怎样把文字变成可比较的向量

语义检索希望理解“针脚图”和“引脚定义”之间的联系。Embedding 模型把一个文本编码为向量；相近含义通常得到较接近的表示。这里的“相近”是模型给出的统计表示，不是业务规则保证的同一型号。

项目默认选择 BAAI/bge-m3，通过 HuggingFaceEmbeddings 接入 LangChain 的 Embeddings 接口。统一接口主要区分 `embed_documents()` 与 `embed_query()`：前者用于准备资料向量，后者用于准备查询向量。项目不需要在业务层手写模型的 tokenizer 和批量推理流程。

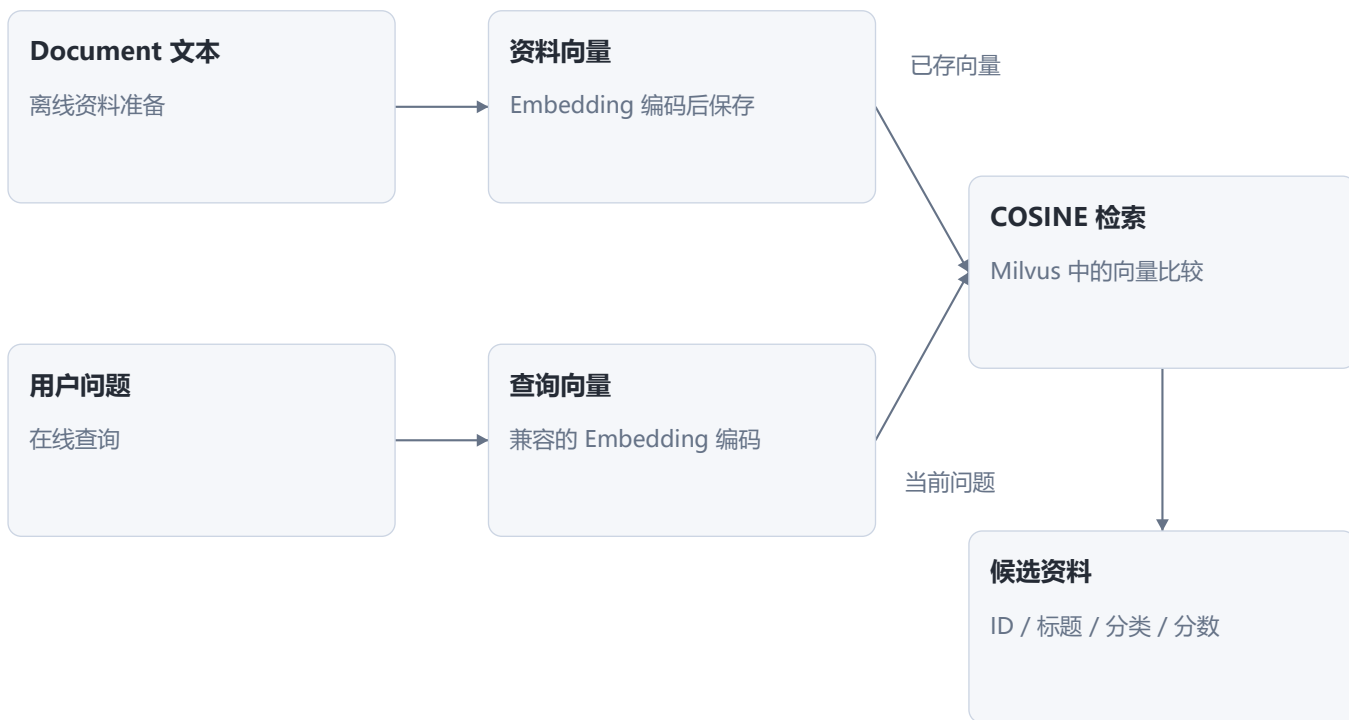


图 2-2. 资料和问题使用兼容的向量模型编码，再按相似度取得候选；相似度不等于业务准确率。

图中的两条编码流程必须保持兼容。资料用一个模型编码，问题用另一个不兼容的模型编码，即使向量维度碰巧相同，相似度也不具有原来的含义。更换 Embedding 模型或明显改变检索文本模板时，应评估旧索引是否需要重新生成。

源码节选：以下是当前模型包装的重要参数，完整函数见 `app/rag/embeddings.py`。

代码块 python

```
1 embeddings = HuggingFaceEmbeddings(  
2     model_name=settings.embedding_model,  
3     model_kwargs={  
4         "device": settings.embedding_device,  
5     },  
6     encode_kwargs={  
7         "batch_size": settings.embedding_batch_size,  
8         "normalize_embeddings": settings.embedding_normalize,  
9     },  
10    query_encode_kwargs={  
11        "normalize_embeddings": settings.embedding_normalize,  
12    },  
13    show_progress=False,  
14 )
```

配置项	源码默认值	学习含义
embedding_model	BAAI/bge-m3	当前向量模型标识
embedding_device	cpu	默认在 CPU 上执行编码
embedding_batch_size	32	资料编码批次大小
embedding_normalize	True	文档和查询都做向量归一化

这些是 Settings 类默认值，实际运行可能被 .env 或环境变量覆盖。批次大小 32 是编码批次，入库批次默认 256 是数据库写入批次，二者解决的问题不同。

进一步理解：BGE-M3 模型自身提供多种表示能力，不等于当前包装同时启用了全部能力。本项目由 HuggingFaceEmbeddings 提供 Dense 向量，Hybrid 中的 Sparse 路径来自 Milvus 内置 BM25，而不是在这段代码里额外取得 BGE-M3 的稀疏输出。

易错点：向量相似可以帮助召回候选，但不能保证型号精确、资料完整或答案正确。相似度分数也不能直接解释为“有 80% 的概率就是用户要的文件”。

2.8 Milvus: Collection、主键与索引参数

Milvus 负责保存可检索的向量与文本，并提供查询能力。它不是负责生成回答的对话模型，也不是保存任意文件后就自动理解图片内容的文件系统。本项目通过 langchain_milvus.Milvus 使用它。

Collection (集合) 可以类比为一组结构相同的记录；**Field (字段)** 表示一条记录的某个属性；**Entity (实体)** 表示一条完整数据。类比有助于入门，但不能直接推断所有关系数据库操作都适用于向量集合。

Dense Collection 默认名是 circuit_documents_dense，Hybrid Collection 默认名是 circuit_documents_hybrid。两个名称来自配置。Dense 存稠密检索所需数据，Hybrid 在同一集合里配置 dense 和 sparse 两种向量字段。



图 2-3. Dense Collection 负责稠密检索；Hybrid Collection 在同一集合里配置 dense 与 sparse 字段，Sparse 由 Milvus 内置 BM25 产生。

配置	当前源码值	需要理解的作用
Dense 索引	HNSW	使用近似近邻图结构查找向量
Dense 度量	COSINE	用余弦相似衡量向量方向关系
M	16	HNSW 图结构相关参数
efConstruction	200	建索引时搜索候选的范围参数
查询 ef	64	查询时搜索候选的范围参数
Sparse 索引 / 度量	AUTOINDEX / BM25	稀疏全文检索的当前配置

读参数时先记清“建立索引”和“执行查询”的阶段差异。更大的搜索范围通常需要更多计算，但当前项目没有提供这些参数的对照实测，不能写成“改成 64 后延迟降低多少”。参数是否适合业务，应该由标注集与延迟记录验证。

auto_id=False 表示项目主动提供主键，而不是交给数据库自动生成。写入时使用 str(doc_id) 作为稳定 ID。enable_dynamic_field=True 允许当前封装保留 metadata 所需字段，但不会替代 Loader 的数据质量检查。

consistency_level 默认是 Session，连接 URI 默认是 http://localhost:19530。这些是连接与读写语义配置，不是运行成功的证据。文件中写着某个 Collection 名，也不能证明对应数据库已经启动或记录已经全部写入。

2.9 Dense 与 Hybrid：同一份资料，两种检索线索

Dense 检索主要利用语义表示；Sparse 检索利用词项及其统计信息。用户写“供电线路”而资料标题写“电源电路”时，语义线索可能有帮助；用户明确给出“4HK1”或“EDC7”这类编号时，词项线索也很有价值。

项目的 Hybrid Collection 使用 BM25BuiltInFunction，把 analyzer 配置为中文，并把输出稀疏字段名指定为 sparse。Milvus 在这个方案中承担文本分析和 BM25 稀疏表示计算。后续检索通过多路召回、RRF 融合和重排使用这些索引，详见下一章。

源码节选：这里只展示 Hybrid 的 BM25 配置，不是一个可以单独执行的数据库任务。

代码块 python

```
1 bm25_function = BM25BuiltInFunction(  
2     analyzer_params={"type": "chinese"},  
3     output_field_names="sparse",  
4 )
```

应用场景：如果以后学习一个包含大量产品编号的客服项目，应分别观察语义改写和精确编号是否能被召回。可以在评估中把“同义描述”和“强编号约束”分组，判断某一条路径是否系统性漏掉某类问题。

Dense 与 Hybrid 使用同一个 build_search_text()，所以比较时不会一边只读标题、一边同时读分类。这个一致性可以减少非算法因素，但它还不能保证评估完全公平：候选数量、过滤方式、重排开关和模型预热仍需保持可比较。

易错点：Hybrid Collection 的“有两个向量字段”与“两次查询结果怎样合并”是两件事。本节只建立存储概念；RRF 的排名融合发生在检索方案里。不要在学习总结里把 Milvus、BM25、RRF 和 Reranker 写成一个不可分辨的大模型。

2.10 写入策略：新增、更新与重建的差别

index_dense_documents() 和 index_hybrid_documents() 都先整理 Document，再把 doc_id 转为字符串主键，最后调用向量库写入。传入空列表时返回 0；batch_size 小于或等于 0 时抛出异常，避免无意义批次。

选择写入方法的代码根据 Collection 是否已经存在：存在时用 upsert，不存在时用 add_documents。测试通过 FakeVectorStore 检查这两条路径，重点验证方法选择和主键传递，不连接真实 Milvus。

源码节选：省略函数外围说明，下面是完整的写入分支本身。

代码块 python

```
1 ids = [str(doc.metadata["doc_id"]) for doc in docs]  
2 vectorstore = create_dense_vectorstore(drop_old=recreate)  
3 write_documents = (  
4     vectorstore.upsert  
5     if vectorstore.client.has_collection(vectorstore.collection_name)  
6     else vectorstore.add_documents  
7 )  
8 write_documents(ids=ids, documents=docs, batch_size=batch_size)
```

Upsert（插入或更新）解决已有 ID 的数据维护问题；Recreate（重建）通过 `drop_old=True` 处理旧集合，会改变现有索引。本章只阅读这部分代码，未执行入库或重建命令。

一个现实问题：CSV 中某个 ID 被删除之后，重新 upsert 其他记录不会自动证明这个旧 ID 已从数据库中删除。完整同步还需要定义删除策略。当前索引函数返回 `len(docs)`，表示本次提交的文档数量，不是“新增条数”“数据库总数”或“每条记录已经逐一回读验证”的统计。

思考顺序：如果某次改动只更新一个标题，稳定 ID 便于更新同一业务对象；如果换了 Embedding 模型，旧向量与新向量可能不兼容，应制定迁移或重建方案；如果源数据大量删改，应检查数据库中的残留记录。三个变化不要使用同一个含糊的“重新导入”描述。

2.11 离线实验：只读取 CSV，观察转换结果

完整可运行命令：已安装本项目依赖的环境中，在根目录执行。该命令只解析本地 CSV，不初始化 Embedding，不调用模型 API，不写数据库。

代码块 powershell

```
1 .venv\Scripts\python.exe -m scripts.ingest --dry-run --sample-size 1
```

本次真实检查结果：Rows: 4233, Valid documents: 4233, Invalid rows: 0。随后输出第一条 Document 的两行检索文本和 metadata。这说明 Loader 对当前 CSV 的转换成功，不能用来证明在线检索效果。

`scripts/ingest.py` 未传 `--dry-run` 时仍保留“Phase 2”阶段提示。当前源码已经另有 Dense 和 Hybrid 的索引实现，所以这个历史提示只说明预览脚本的使用方式，不能据此判断整个项目还没实现 Milvus。

完整可运行实验：在项目根的 Python 交互终端执行，或者自行保存到临时文件后执行。PDF 不附带学习脚本。

代码块 python

```
1 from app.rag.loader import load_circuit_documents
2
3 result = load_circuit_documents()
4 ids = [doc.metadata["doc_id"] for doc in result.documents]
5 print("有效记录:", result.valid_rows)
6 print("无效记录:", result.invalid_rows)
7 print("唯一 ID:", len(set(ids)))
8 print("重复数量:", len(ids) - len(set(ids)))
9 first = result.documents[0]
10 print(first.page_content)
11 print(first.metadata["hierarchy_segments"])
```

结果解读：本次唯一 ID 是 4233，重复数量是 0；第一条路径片段列表有五项。若以后重复数量大于 0，应先检查源数据再写库。若有效记录为 0，上面直接取 [0] 会失败，应先处理无数据情况；这个实验针对当前非空 CSV。

扩展练习：不用初始化模型，就可以统计各层级深度、最常见的路径前缀或标题缺失情况。这样能先掌握资料分布，再决定查询过滤字段。注意统计必须来自文件，不要把几个示例记录推断为全部资料的结构。

2.12 离线实验：用替身观察 Dense Retriever 的接口

需求是看懂 `dense_search()` 如何接收问题并转为统一结果，而不等待模型加载和数据库连接。函数提供 `vectorstore` 参数，允许注入符合接口的对象，这是一个值得学习的工程边界。

完整可运行教学实验：下方的记录与分数是人为构造的教学数据。本次实际执行了这段替身流程，未调用真实 Milvus。

代码块 python

```
1 from langchain_core.documents import Document
2 from app.rag.retriever import dense_search
3
4 class FakeVectorStore:
5     def similarity_search_with_score(self, query, *, k, expr=None):
6         print(query, k, expr)
7         doc = Document(
8             page_content="示例检索文本",
9             metadata={
10                 "doc_id": 2,
11                 "title": "教学标题",
12                 "hierarchy_path": "电路图->教学分类",
13             },
14         )
15         return [(doc, 0.8)]
16
17 results = dense_search(
18     " 仪表针脚 ", top_k=1, vectorstore=FakeVectorStore()
19 )
20 print(results[0].model_dump())
```

真实执行现象：替身收到 仪表针脚、1、None，说明查询两侧空格已去掉、TopK 已传入、没有附加表达式。输出结果包含 `doc_id`、`title`、`hierarchy_path`、`content`、`score`。其中 0.8 是替身手写的分数，不是这份真实资料的相似度。

易错点：`dense_search()` 当前使用 `top_k = top_k or settings.dense_top_k`。因此传入 0 会落回默认值，而负数才会进入随后的小于等于 0 校验。这是阅读源码时发现的边界行为，不应该在学习材料里改写成“所有非正数一定报错”。

本节小结：替身实验验证接口和转换逻辑；真实检索实验验证模型、索引和业务结果。两种实验对应不同证据，不能互相替代。

2.13 源码阅读与本章检查清单

阅读入口	需要回答的问题	能够留下的证据
app/schemas/document.py	合法记录有哪些字段约束？	字段与校验说明
app/rag/loader.py	一行数据怎样成为 Document？	真实预览与问题统计
app/rag/embeddings.py	文档和问题怎样编码？	模型与参数配置
app/rag/vectorstore.py	Dense 怎样建立和写入？	主键、索引、写入流程
app/rag/hybrid_vectorstore.py	BM25 与 Dense 怎样共存？	两字段配置与职责
tests/test_vectorstore_indexing.py	插入与更新如何被验证？	替身测试覆盖的分支

请用自己的语言回答：为什么路径也放进检索文本？为什么不把每个路径片段做成独立文档？为什么 ID 要稳定？更换向量模型后旧索引有什么风险？为什么资料数 4233 不能证明 Recall@5 很高？回答这些问题，比只记住库名更接近项目理解。

动手任务：先运行只读预览；再检查 ID 唯一性；最后用替身理解 Retriever 返回结构。完成后画出“CSV→Record→Document→Embedding→Collection”的链路，并分别标出每一步新增或删除的字段。下一章将继续研究问题怎样经过改写、混合召回和重排取得候选。

本次工程验证：在项目现有虚拟环境中运行 Loader、向量写入替身、指标函数和 Benchmark 过滤四组既有离线测试，共 12 项通过。其中向量写入测试注入 FakeVectorStore，没有真实建库；它证明分支选择和主键传递符合测试，不能证明服务连接、索引召回或数据库持久化成功。

2.14 排查练习：先确定问题发生在哪一层

遇到“新增资料查不到”，先确认 CSV 中存在记录、Loader 认为有效、ID 没有重复，再核对服务连接的 Collection 和写入状态。

如果只有完整标题才能命中，对比查询文本、Dense、BM25 和过滤表达式。完整标题命中不能证明同义检索正常；相似条目很多也不能证明型号约束已满足。

如果 CSV 和 Document 已更新，页面仍显示旧标题，核对索引、Collection 与缓存。get_document_index() 和模型构造函数使用进程内缓存，修改源文件不会自动刷新已构造对象。

按“源数据→转换→存储→查询→展示”留下证据。例如，“Loader 成功，但目标 ID 未进入无过滤召回”比“RAG 不准”更容易定位。学习其他项目时也可以沿这条链路排查。

本章小结：数据加载保证输入可用，Document 保证表示统一，Embedding 提供语义比较，Milvus 提供索引与存储。可靠的检索还需要查询处理、排序、过滤与评估，这些将在后续章节中连接起来。

延伸阅读：原飞书资料《第1节 RAG Agent》《第4节 Milvus向量库》的相关概念；[BGE-M3 官方模型卡](#)；[Milvus 官方全文检索说明](#)。模型能力以实际包装调用为准，项目行为以当前源码为准。